

Unit – III

Dr. P Arunkumar

Hadoop & MapReduce

Storing and Processing Big Data

The Problem: "Big Data"

- How do you process 10 Terabytes (TB) of data on a single computer? **You can't.**
- **Volume:** The data is too large to fit on one machine's disk.
- **Velocity:** The data is generated too quickly for one machine to process.
- **The Solution:** Distribute the problem. Use a **cluster** of hundreds or thousands of computers working together in parallel.



The Solution: Apache Hadoop

- An **open-source framework** by Apache for storing and processing massive datasets.
- Designed to scale from a single server to **thousands of machines**.
- Provides **distributed storage** and **distributed processing**.
- Built on the principle of using **commodity hardware** (clusters of standard, inexpensive computers).

Hadoop's Core Components

1. HDFS (Storage)

Hadoop Distributed File System

A distributed file system designed to run on commodity hardware. It's the **storage layer** of Hadoop, responsible for holding the massive datasets.

It answers: "How do we store the data?"

2. MapReduce (Processing)

Programming Model

A programming model for processing large datasets in parallel. It's the **processing layer** of Hadoop, responsible for running computations on the data.

It answers: "How do we process the data?"

Part 1: HDFS



The Storage
Layer

HDFS Architecture

NameNode (Master)

The "Table of Contents" for the file system.

- Stores all **metadata** (e.g., file names, permissions).
- Knows which **DataNodes** hold the blocks for a specific file.
- A critical component for the cluster's health.

DataNode (Worker)

The "Storage" for the actual data.

- There are many DataNodes in a cluster.
- They **store the actual data blocks** (e.g., 128MB chunks).
- They report back to the NameNode with "heartbeats".

HDFS Key Feature: Replication

Built-in Fault Tolerance

Hadoop is designed for a world where hardware **will** fail.

Files are split into large **blocks** (e.g., 128MB).

By default, each block is replicated **3 times** and stored on different DataNodes, often in different server racks.

If a node (or even a whole rack) fails, the data is not lost and can be accessed from another copy.

Part 2: MapReduce

The Processing
Model

What is the MapReduce Model?

-  A **Programming Model** for parallel data processing.
-  It uses a "**Divide and Conquer**" strategy to handle massive datasets.
-  **Key Insight:** Instead of moving massive data to the computation, **move the computation to the data**.
-  As a developer, you simplify your complex logic into two simple functions: **Map** and **Reduce**.

The Programming Model: Two Functions

1. The MAP Function

Job: Filter & Transform

The Mapper's job is to take a piece of the input data, perform an operation on it (like filtering or formatting), and output intermediate **(key, value)** pairs.

Example: (Input line) -> (word, 1)

2. The REDUCE Function

Job: Aggregate & Summarize

The Reducer's job is to receive a single key and the **list of all values** associated with that key, and "reduce" them into a single output.

Example: (word, [1, 1, 1]) -> (word, 3)

The Power of the Framework



Parallelization

The framework automatically splits the data and runs thousands of Map tasks in parallel across the cluster.



Fault Tolerance

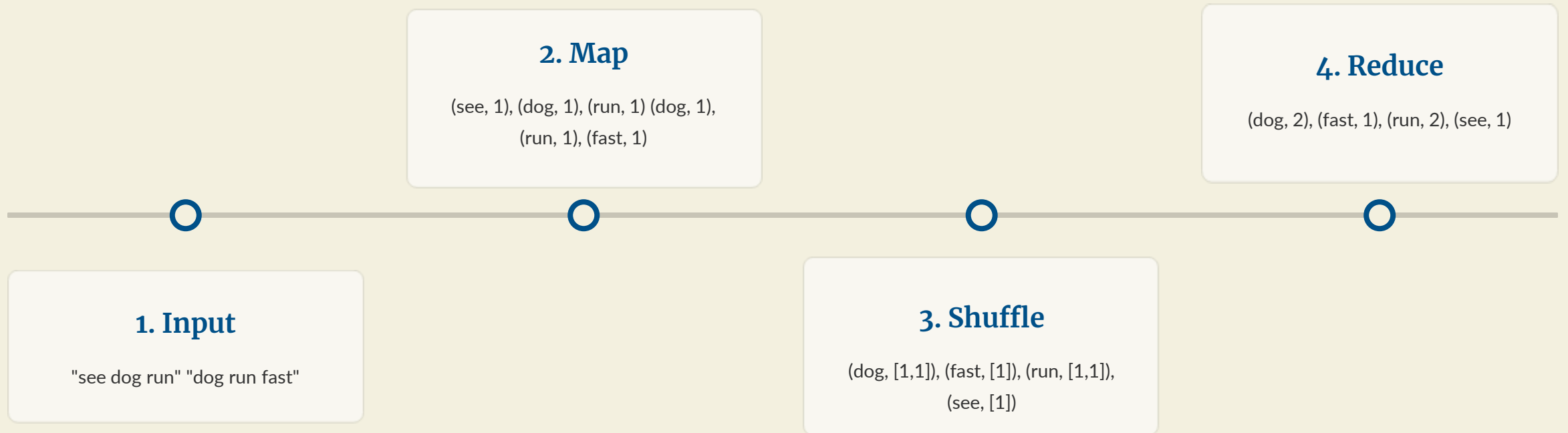
If a machine (or "node") fails, the framework automatically detects it and re-runs its task on a healthy machine.



Data Shuffling

The complex and critical process of moving and sorting all intermediate data from the Mappers to the correct Reducers.

Example: The "Word Count" Flow



Part 3: The Modern Ecosystem

YARN and
Beyond

The Evolution: YARN (Hadoop 2.0)



The "Cluster OS"

YARN (Yet Another Resource Negotiator) is the cluster's resource manager. Think of it as the "brain" or "OS" of the cluster.



Decoupling

YARN's job is to manage cluster resources (CPU, RAM). It separates resource management from the MapReduce processing logic.



Beyond MapReduce

Because of YARN, other processing engines (like Apache Spark, Tez, or Flink) can run on top of HDFS, not just MapReduce.

Modern Hadoop Architecture

The modern Hadoop ecosystem is layered. **HDFS** provides the storage. **YARN** manages the cluster resources. On top, various processing engines like **MapReduce** (for batch), **Spark** (for in-memory), and **Hive** (for SQL) can run simultaneously.

Key Takeaways



Hadoop = Layers

Hadoop is an ecosystem. At its core is **HDFS (Storage)** and **YARN (Resource Management)**.



MapReduce = A Model

MapReduce is the original **processing model** that runs on Hadoop, simplifying parallel logic for developers.



Fault Tolerance is Key

The entire system (HDFS replication, MapReduce task re-tries) is designed from the ground up to expect and handle hardware failure.